Background:

You are working on an e-commerce platform with products and customers managed by other teams which you can read from an API (See OpenAPI spec in *`Billing - e-commerce Interview - API.json`*). Customers and products can change over time, outside of your control (updating addresses, updating prices, descriptions, etc.). You're responsible for building the purchase flow for the platform.
There's also a Shipment API to create a new shipment of the product to the customer's home.

For simplicity, customers pay for purchases with a credit balance that is pre-assigned, but it's your responsibility to keep track of this balance throughout their purchases.

Description:

In **Typescript**, design and implement a simple e-commerce server that exposes the following operations:

- Grant/deduct credit balance
- Get a customer's credit balance
- Purchase a product
    - As part of this operation, call CreateShipment to send the customer the purchased product.
    - Consider if other changes should persist in case the CreateShipment call fails.
- List product purchases
- Refund a purchase (fully or partially)

Design the data structure for the purchase and credit entities (and any other additional entities you see fit) and manage them end-to-end.

For this implementation, it's easiest to mock the APIs that provide Customer, Products and Shipments.

Notes:

- Use best practices, tools and design patterns that you see fit. Avoid `any` types. Functions need strict input and output typings.
    - API requests and responses should have explicitly defined interfaces.
- Since this system handles billing data, consider the data model's auditability and the importance of historical record keeping. For example: the ability to tell when credit balances changed and for what reason.
- Use a DB for this implementation: we're looking to see how you handle data models and application-DB integration.
- If any additional components are required to run your code (databases, message brokers, etc.), please include a docker or docker compose file to run them in a container.
    - Assume node.js, npm, yarn and docker are installed

- Your thought process matters. If you have to ignore or sidestep something you'd normally address, please explain why you did and how you'd approach it in real life.
- Show how you work. Create a git repository and commit your changes as you go. You can create a repository locally by running `git init` in the root directory of the project. Include the .git directory with your code.
- Using an LLM to generate parts of the code is encouraged, but the resulting code should still be structured and readable. Excessive complexity and overwhelming amounts of documentation aren't welcome. Use whatever tools you want, but keep it maintainable (we'll be asking you to add features and make changes in the follow-up interview).
    - Please provide any skill or other agent files used.
- Bonus tasks are a way to showcase your ability and familiarity with additional aspects of the problem space. Prioritize good implementation over a full implementation riddled with issues.

Bonus:

- Implement a simple internal react application (feel free to use ClickUI) for a customer service rep to:
    - Navigate through customers' purchases
    - Refund purchases
    - Manage customers' credit balance
    - This might require implementing additional APIs
- Cache customers and products to improve performance without compromising accuracy of details consumed.
- Include e2e tests